

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY (AUST)

141 & 142, Love Road, Tejgaon Industrial Area, Dhaka-1208.



Department of Computer Science and Engineering
Program: Bachelor of Science in Computer Science and Engineering

Project Final Report

Course No: CSE-1200

Course Title: Software Development – I

Section: B, Group: 04

Project Title: KeyScape

Date of Submission: 29/09/26

Submitted to

Mr. Saha Reno

Assistant Professor, Department of CSE, AUST.

Mr. Md. Zahid Hossain

Lecturer, Department of CSE, AUST.

Submitted by,

Name: Tanusree Saha

Student ID: 00725105101062

Name: Sumia Haque Snigdha

Student ID: 00725105101065

Name: Wahidunnahar Bhuiyan Nazah

Student ID: 00725105101068

Introduction

1.1 Project Overview

KeyScape is a 2D side-scrolling adventure game developed using the iGraphics library in C/C++. The player runs, jumps, and slides across three distinctly themed worlds — a desert, an undersea trench, and a cave system — collecting keys and treats along the way. Each level ends at its own bank of doors: Level 1 has 3, Level 2 has 4, and Level 3 has 5, and every door hides a self-contained challenge (a puzzle, a boss fight, or a survival minigame) that must be solved before the player can move on.

1.2 Objective

- To develop an interactive 2D side-scrolling game using the iGraphics library.
- To implement player movement mechanics (running, jumping, sliding) along with collision detection.
- To design three distinctly themed levels, each with its own obstacles, collectibles, and door-based finale.
- To build a variety of self-contained door challenges — memory puzzles, timed tasks, a phase-based boss fight, and survival minigames — each with its own win/lose logic.
- To add scoring, energy/lives systems, background music, sound effects, and a persistent high-score record for each level.

Game Development

2.1 Tools & Technologies Used

- **Programming Language:** C, C++
- **IDE:** Visual Studio 2013
- **Graphics Library:** iGraphics
- **Platform:** Windows PC

2.2 Core Features

- **Player Movement:** Run, jump, and slide across every level and door to dodge obstacles and hazards.
- **Home Page & Navigation:** Main menu with Play, Options, Highscore, and Credits pages, plus an Easy/Medium/Hard difficulty select before each level.
- **Scoring & Lives:** Points for collectibles and survival, energy/heart-based life systems, and a per-level persistent high score saved to disk.

- **Pause & Settings Menu:** In-game popup for resume, restart, mute, and returning to the main menu, available inside every level.
- **Game Over Condition:** When the energy bar of the character hits zero, then the game is over.

Implementation Details

3.1 Code Structure

The project is organized into modular header files, each owning one screen or one gameplay system:

- `iMain.cpp`: Entry point — the `iDraw/fixedUpdate` switch, mouse and keyboard routing, and which background track plays for the current game state.
- `homepage.h`, `playbutton.h`, `option.h`, `score-board.h`, `credit.h`: The main menu, the Easy/Medium/Hard difficulty-select screen, the sound/settings popup, the persisted high-score page (backed by the `*_highscore.txt` files), and the credits page.
- `Level1.h`: The desert run — cactus spawning, the energy/score HUD, the 3-door branch, the rune-lock memory puzzle, the treasure box, and the Scorpion/Mummy duel.
- `Level2.h` + `L2doors.h`: The undersea run — obstacle and 4-key spawning along a 6,800-unit track, the 4-door hub with its Math/Puzzle/Color tasks and decoy-key logic, and the phase-based Guardian boss fight.
- `Level3.h`: The shared jungle run — the 6 elevated “exclave” platforms, the 5-key and 24-treat spawn tables across a 22,500-unit track, the four power-up types, and the 5-door hub that hands control off to `L3door1.h` – `L3door5.h`.
- `L3door1.h` (Door 1): A 40-second bottle-dodging survival minigame — poison bottles damage the player across three lanes, medicine bottles heal or grant a timed shield, and a floor hole opens roughly every 7 seconds.
- `L3door2.h` (Door 2): A quota-based coin-collection minigame — the player must gather set numbers of Bronze, Silver, and Gold coins while dodging a rolling bomb.
- `L3door3.h` (Door 3): A free-roam, dimly lit crystal-cave minigame — survive 60 seconds and collect 3 crystals (plus a rare speed-boosting special crystal) while dodging falling rocks and fighting off roaming scorpions.
- `L3door4.h` (Door 4) — “Cave Storm”: A 60-second survival minigame against falling rocks and spikes, using only left/right movement.
- `L3door5.h` (Door 5): A floating-steps minigame — timed jumps across sliding steps, ending in a sum-lock puzzle and a treasure-chest win sequence.
- `iGraphics.h`, `glut.h`, `glaux.h`, `bitmap_loader.h`, `stb_image.h`: The graphics/windowing library and the image-decoding utilities everything above is built on.

3.2 Challenges Faced & Solutions

Challenge	Solution
Obstacles and collectibles were too predictable or too close together	Used randomized gaps and fixed spawn-distance tables
Music restarted and levels reset every frame	Used state flags so setup and music start only once, and moved game logic into fixedUpdate()
Merge conflicts while teammates worked on the same project	Kept each level and door in its own header file and merged through GitHub branches

Screenshots



Figure 1: Home Page

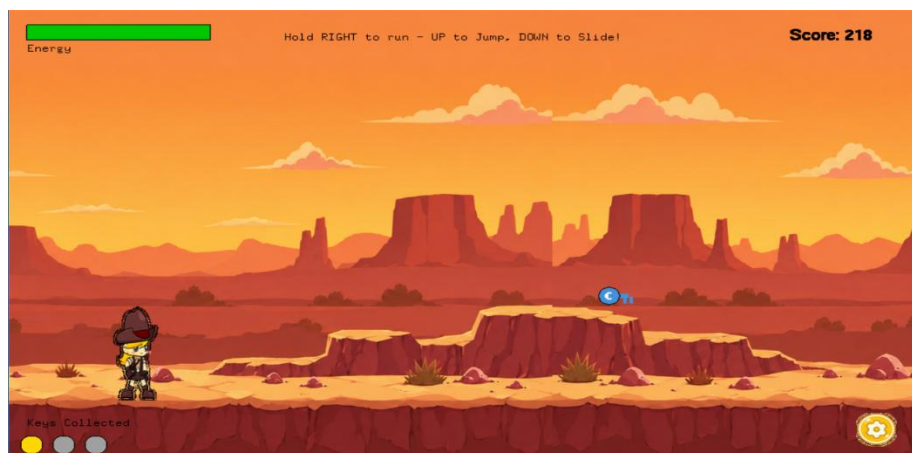


Figure 2: Level 1 – Desert Escape



Figure 3: Level 2 – Undersea Trial

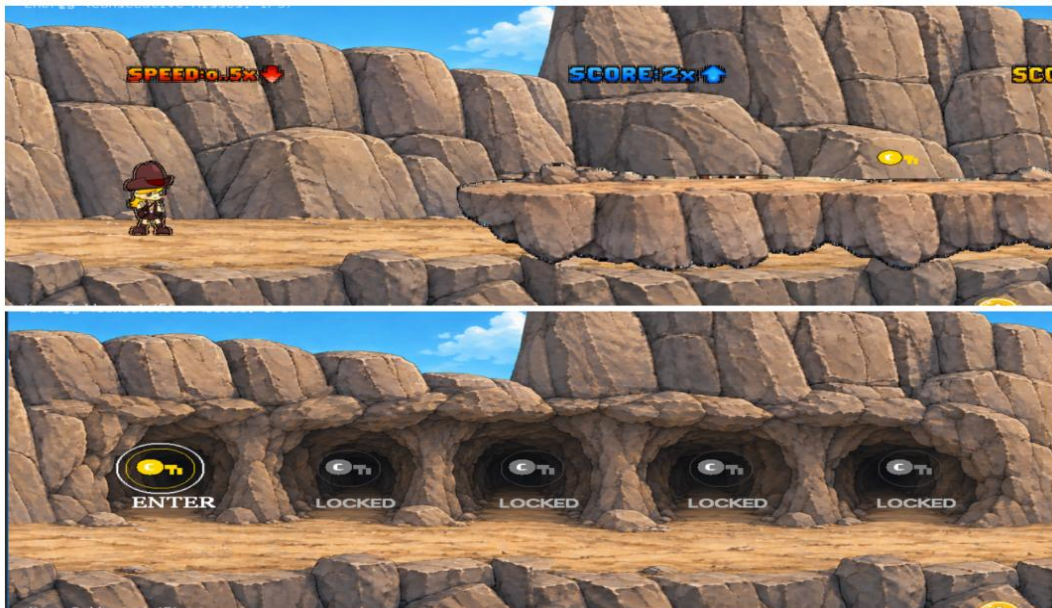


Figure 4: Level 3 – The Five Doors



Figure 5: Door Minigame

GitHub Collaboration

4.1 Repository Management

The project was managed using GitHub (repository: KeyScape) to enable efficient collaboration, version control, and code sharing among team members. A dedicated repository was created where all source code, image/audio assets, and documentation were maintained.

Each team member contributed through:

- Creating and updating files via commits
- Working on separate modules (levels/doors) using branches
- Merging changes using pull requests
- Resolving conflicts collaboratively

4.2 Contribution Workflow

Task Distribution → Individual Development → Code Review → Integration → Testing → Bug Fixing

In practice, this broke down into the following steps:

1. Repository initialized with the project structure
2. Members cloned the repository to their local systems
3. Features (levels and doors) were developed in separate branches
4. Changes were committed with meaningful messages

5. Pull requests were created and reviewed
6. Approved changes were merged into the main branch

This workflow ensured:

- Proper tracking of contributions
- Reduced code conflicts
- Better team coordination

4.3 Individual Contributions

Member	Checkpoint 01	Checkpoint 02	Checkpoint 03	Main Areas
Tanusree Saha (ID: 00725105101062)	35%	40%	33.33	Menu, Score, Sound, Level 2 path, Level 3 core gameplay (Level3.h), Door 1 minigame (L3door1.h)
Sumia Haque Snigdha (ID: 00725105101065)	35%	30%	33.33	Level 1 path, Level 2 doors (Puzzle and combat), Level 3 Door 4 minigame (L3door4.h), Door 5 minigame (L3door5.h)
Wahidunnahar Bhuiyan Nazah (ID: 00725105101068)	30%	30%	33.33	Level 1 doors (puzzle and combat), Level 3 Door 2 coin-collection minigame (L3door2.h), Door 3 crystal-cave minigame (L3door3.h)

4.4 Contribution Evidence



Figure 6: GitHub Contribution Graph

Challenges & Limitations

5.1 Challenges

- iGraphics calls iDraw() every frame. Starting the background music or calling setupLevel() inside it restarted the music and reset the level over and over. We fixed this with flags (isBgMusicPlaying) and a lastGameState check, so music and setup run only once.
- iGraphics has no fixed-timestep loop, so movement speed depended on how fast frames were drawn. We moved movement and collision into fixedUpdate() (a 20 ms timer) and left iDraw() for drawing only.
- The three levels and five doors were built by different teammates and differ a lot in length. We reused the same jump, gravity and slide constants across levels and doors to keep the movement feel consistent.

5.2 Limitations

- Each door has its own separate logic, so a change to one door does not carry over to the others.
- High scores are saved per level in text files, so there is no single save system.
- The game runs only on Windows with Visual Studio.

Conclusion & Future Improvements

6.1 Summary

KeyScape set out to build more than a single-track runner: a three-world 2D adventure — desert, undersea, and jungle cave — where the core run/jump/slide mechanic leads into a bank of doors at the end of every level. Those doors hold a wide variety of challenges: a memory puzzle and enemy duels in Level 1, math, sliding-puzzle, and color tasks plus a telegraphed boss fight in Level 2, and five independent survival, collection, and puzzle minigames in Level 3. It was built entirely on the iGraphics/GLUT library in C/C++ by a three-person team. Separating fixed-timestep game logic from continuous rendering, and giving each level and door its own header file, kept the codebase workable as the number of screens grew.

6.2 Future Enhancements

- Additional Doors/Levels
- Save/Load Progress
- Improved Visuals
- Cross-platform Support